

Stajyer Gelişim Programı

StajDesk: Destek Talebi Yönetim Sistemi

16 hafta | Haftada 3 gün (Çarşamba, Perşembe, Cuma) | 48 staj günü

Docker | .NET | PostgreSQL | Redis | RabbitMQ | Vue.js

Stajyer

Mentor

Başlangıç tarihi

Bitiş tarihi

Amaç: Kullanılan her teknolojinin ne olduğunu, neden seçildiğini ve nasıl birlikte çalıştığını, gerçek bir uygulamayı sıfırdan kendi ellerinizle geliştirerek öğrenmek.

Hoş geldiniz

Devoplus yazılım ekibine hoş geldiniz. Önümüzdeki yaklaşık dört ay boyunca, bir şirketin gerçekten kullanabileceği bir web uygulamasını baştan sona kendiniz geliştireceksiniz. Bu dokümanda hangi gün ne öğreneceğiniz, ne yapacağınız ve günün sonunda neyi teslim edeceğiniz adım adım yer almaktadır.

Bu programın hedefi yalnızca "çalışan bir uygulama" değildir. Asıl hedef, kullandığınız her teknolojinin **ne olduğunu, hangi problemi çözdüğünü ve neden seçildiğini** kendi cümlelerinizle anlatabilmenizdir. Bu yüzden her haftanın sonunda cevaplamanız gereken kavrama soruları bulunmaktadır.

Çalışma düzeni

Konu	Açıklama
Staj günleri	Çarşamba, Perşembe, Cuma (haftada 3 gün)
Süre	16 hafta = 48 staj günü. Resmî tatiller ve sınav dönemleri planı bir hafta kaydırır; atlama yapılmaz.
Günün akışı	Güne mentorla 10 dakikalık kısa bir planlama ile başlanır. Günün ilk yarısı öğrenme (okuma, deneme, küçük örnekler), ikinci yarısı uygulama (proje görevi) içindir. Gün, LOG.md kaydı ve commit ile kapanır.
Cuma değerlendirmesi	Her cuma günü sonunda mentorla 30 dakikalık görüşme: haftanın demosu, kavrama soruları, kod incelemesi ve bir sonraki haftanın hazırlığı.

Temel kurallar

- Yapay zekâ araçları kullanılmaz.** ChatGPT, Claude, Copilot, Gemini vb. araçlarla kod veya cevap üretilmez; IDE'deki yapay zekâ destekli kod tamamlama eklentileri kapalı tutulur. Amaç sizin öğrenmenizdir; hazır cevap bu amacı ortadan kaldırır.
- Serbest kaynaklar:** Resmî dokümantasyonlar, kitaplar, eğitim videoları ve forumlar (ör. Stack Overflow) okunabilir. Ancak kod kopyalanıp yapıştırılmaz; anladığınızı kendiniz yazarsınız.
- 20 dakika kuralı:** Bir konuda takıldığınızda en az 20 dakika kendiniz uğraşınız (dokümana bakın, hatayı okuyun, deneyin). 20 dakika sonra hâlâ ilerleyemiyorsanız mentora ne dendiğinizi anlatarak sorunuz. Soru sormak zayıflık değil, işin bir parçasıdır.
- Öğrenme günlüğü:** Her gün sonunda **LOG.md** dosyasına üç madde yazınız: bugün ne öğrendim, nerede takıldım ve nasıl çözdüm, bir sonraki staj günü ne yapacağım.
- Her gün en az bir commit:** Yaptığınız her şey Git deposunda görünür olmalıdır.
- Gizli bilgi paylaşılmaz:** Şifre, anahtar ve bağlantı bilgileri koda veya depoya yazılmaz.

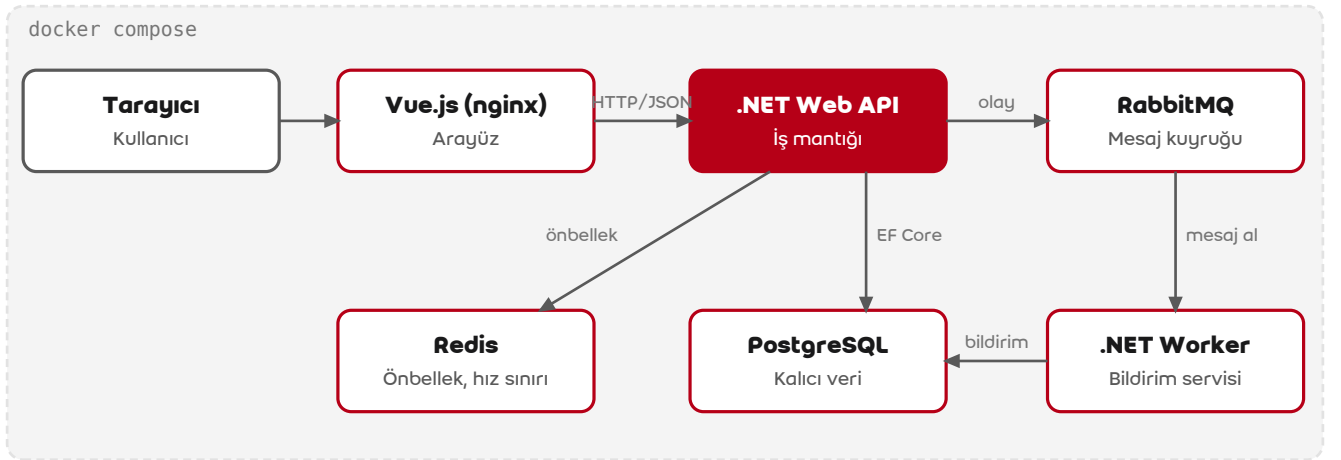
Proje: StajDesk

StajDesk, bir şirketin destek birimine gelen talepleri yöneten bir web uygulamasıdır. Kullanıcılar sisteme kaydolur, bir sorunla karşılaştığında talep açar; destek personeli bu talepleri görür, yanıtlar ve durumlarını günceller. Devoplus'ın günlük işinde de benzer sistemler kullanılmaktadır, bu nedenle geliştireceğiniz her parça gerçek iş dünyasındaki bir karşılığa sahiptir.

Uygulamanın temel özellikleri

- Kayıt olma, giriş yapma ve rol bazlı yetki (Kullanıcı, Destek Personeli, Yönetici)
- Talep açma, listeleme, filtreleme, yorum ekleme ve durum takibi (Açık → İşlemde → Çözüldü → Kapandı)
- Kategori yönetimi ve talep sayılarını gösteren özet paneli
- Talep açıldığında veya durumu değiştiğinde arka planda oluşturulan bildirimler
- Tüm sistemin tek komutla (`docker compose up`) çalıştırılması

Mimari



Kullanıcı tarayıcıdan Vue arayüzüne erişir. Arayüz, verileri .NET Web API'den JSON olarak ister. API kalıcı veriyi PostgreSQL'de tutar, sık okunan veriyi Redis'te önbelleğe alır. Bir talep açıldığında API bunu RabbitMQ'ya bir mesaj olarak bırakır; Worker servisi bu mesajı alır ve bildirimi oluşturur. Tüm parçalar Docker konteynerlerinde çalışır.

Hangi teknoloji ne işe yarar?

Bir web uygulamasını bir restorana benzetirsek, her teknolojinin görevi aşağıdaki gibidir. Program boyunca her teknolojiyi öğrendiğiniz haftada bu tabloya geri dönüp kendi benzetmenizi eklemeniz beklenir.

Teknoloji	Benzetme	Görevi ve neden seçildi
Vue.js	Restoranın salonu ve menüsü	Kullanıcının gördüğü ve tıkladığı arayüz. Bileşen yapısı sayesinde ekranlar küçük, yeniden kullanılabilir parçalardan oluşur; veri değişince ekran kendiliğinden güncellenir (reaktivite). Öğrenmesi görece kolay olduğu için ilk frontend çatısı olarak uygundur.
.NET (C#)	Mutfak ve şef	Tüm iş kurallarının uygulandığı yer: kim hangi talebi görebilir, bir talep hangi durumlara geçebilir. Güçlü tip sistemi hataları erken yakalar, performansı yüksektir ve kurumsal dünyada yaygındır. Devoplus'ın ana backend teknolojisidir.
PostgreSQL	Kiler ve arşiv defteri	Kalıcı verinin tutulduğu ilişkisel veritabanı. Sunucu kapansa da veri kaybolmaz. Açık kaynaklıdır, güvenilirdir (ACID) ve tablolar arasındaki ilişkileri (kullanıcı → talep → yorum) doğal biçimde modeller.
Redis	Şefin tezgâhtaki hazır malzemeleri	Bellekte çalışan çok hızlı bir anahtar-değer deposu. Her seferinde kilere gitmek yerine sık kullanılan veri el altında tutulur. Geçicidir; asıl kaydın yerini tutmaz, onun önünde hız kazandırır. Ayrıca sayaç tutmak (ör. giriş denemesi sınırı) için idealdir.
RabbitMQ	Mutfaktaki sipariş fişi askısı	Garson siparişi askıya bırakır ve diğer masaya geçer; aşçı fişi hazır olduğunda alır. Böylece API, bildirim gönderme gibi yavaş işleri beklemeden kullanıcıya hızla yanıt verir. Worker kapalı olsa bile fişler (mesajlar) kaybolmaz.
Docker	Standart yük konteyneri	Her parçayı gerekli her şeyiyle birlikte paketler. Uygulama sizin bilgisayarınızda, mentorunuzun bilgisayarında ve sunucuda aynı şekilde çalışır. docker compose ile tüm restoran tek komutla açılır.

16 Haftalık Yol Haritası

Program, her biri bir öncekinin üzerine inşa edilen aşamalardan (faz) oluşur. Her aşamanın sonunda uygulama bir adım daha büyür; hiçbir hafta "sadece teori" değildir.

Hafta	Aşama	Konu	Hafta sonu çıktısı
1	Temeller	Başlangıç, Git ve Web'in Çalışma Mantığı	HTTP istek/yanıt akışını kendi cümlelerinizle anlatabiliyorsunuz.
2	Temeller	Docker ile Tanışma	Çalışan compose dosyası ve el çizimi mimari şema.
3	C# ve .NET	C# Temelleri	LINQ alıştırması tamam; her sorgunun ne yaptığını açıklayabiliyorsunuz.
4	C# ve .NET	Nesne Yönelimli Programlama (OOP)	StajDesk v0 (konsol) depoda.
5	C# ve .NET	ASP.NET Core Web API	Katmanlı API + Postman koleksiyonu depoda.
6	PostgreSQL	İlişkisel Veritabanı ve SQL	Onaylanmış ER diyagramı depoda (/docs).
7	PostgreSQL	Entity Framework Core ile API'yi Veritabanına Bağlamak	API artık kalıcı veriyle çalışıyor.
8	Güvenlik	Doğrulama, Hata Yönetimi ve Kimlik Doğrulama	Güvenli ve rol bazlı çalışan API.
9	Vue.js	Frontend Temelleri ve Vue.js'e Giriş	Bileşen tabanlı liste sayfası.
10	Vue.js	Vue Router, Pinia ve API Bağlantısı	Oturum yönetimi tamam.
11	Vue.js	Uygulama Ekranları	StajDesk uçtan uca çalışıyor (MVP).
12	Redis	Redis ile Önbellekleme ve Hız Sınırlama	Giriş ucu brute-force'a karşı korumalı.
13	RabbitMQ	RabbitMQ ile Asenkron Mesajlaşma	Asenkron bildirim sistemi çalışıyor.
14	Bütünleştirme	Tüm Sistemin Docker ile Çalıştırılması	Tek komutla ayağa kalkan sistem.
15	Bütünleştirme	Kalite: Test, Loglama ve Dokümantasyon	Belgelenmiş, test edilmiş proje.
16	Kapanış	Son Dokunuşlar ve Final Sunumu	Staj tamamlandı.

Kilometre taşları

Hafta 4: Konsolda çalışan ilk sürüm · Hafta 7: Veritabanına bağlı API · Hafta 11: Uçtan uca çalışan uygulama (MVP) · Hafta 13: Redis ve RabbitMQ entegre · Hafta 14: Tek komutla çalışan sistem · Hafta 16: Final sunumu

FAZ 0 · TEMELLER

Hafta 1 | Başlangıç, Git ve Web'in Çalışma Mantığı

Haftanın hedefi: Geliştirme ortamını kurmak, Git ile çalışmaya başlamak ve bir web isteğinin arka planda nasıl işlediğini kavramak.

Çarşamba | Gün 1/48

Tanışma ve ortam kurulumu

Tarih: ____/____

ÖĞRENİN

- Stajın amacı, StajDesk projesinin tanıtımı ve 16 haftalık yol haritası
- Bir yazılım ekibi nasıl çalışır: görev takibi (Jira), sürüm kontrolü (Git), kod incelemesi
- Kullanılacak araçlar ve ne işe yaradıkları

UYGULAYIN / ÖDEV

- ☐ VS Code, Git, .NET SDK, Node.js (LTS) ve Docker Desktop kurulumlarını yapınız
- ☐ `dotnet --version, node -v, git --version, docker --version` çıktılarını not ediniz
- ☐ Masaüstünde `stajdesk` klasörü açıp içine `LOG.md` (öğrenme günlüğü) dosyası oluşturunuz

☐ **Gün sonu çıktısı:** Tüm araçlar kurulu ve çalışıyor; LOG.md'de ilk günlük kaydı var.

Perşembe | Gün 2/48

Terminal ve Git temelleri

Tarih: ____/____

ÖĞRENİN

- Terminal komutları: `cd, ls/dir, mkdir`, dosya yolu (görelî/mutlak)
- Git nedir, neden kullanılır: repo, commit, branch, merge, remote
- İyi bir commit mesajı nasıl yazılır

UYGULAYIN / ÖDEV

- ☐ Şirket Git sunucusunda `stajdesk` deposunu oluşturup bilgisayarınıza klonlayınız
- ☐ README.md yazınız (kendinizi ve projeyi tanıttığınız), en az 5 anlamlı commit atınız
- ☐ Bir `deneme` branch'i açıp değişiklik yapınız ve ana dala merge ediniz

☐ **Gün sonu çıktısı:** Depo linki mentorla paylaşıldı; commit geçmişi düzenli.

Cuma | Gün 3/48

Web nasıl çalışır? (HTTP ve JSON)

Tarih: ____/____

ÖĞRENİN

- İstemci-sunucu modeli, URL yapısı, DNS'e kısa bakış
- HTTP metotları (GET, POST, PUT, DELETE) ve durum kodları (200, 201, 400, 401, 404, 500)
- JSON veri formatı; tarayıcı geliştirici araçlarında Network sekmesi

UYGULAYIN / ÖDEV

- ☐ 3 farklı sitede Network sekmesinden istekleri inceleyip LOG.md'ye tablo hâlinde yazınız (metot, adres, durum kodu)
- ☐ Postman veya Bruno ile `jsonplaceholder.typicode.com` üzerinde GET ve POST istekleri gönderiniz
- ☐ Cuma değerlendirmesi: haftanın özetini mentora 5 dakikada anlatınız

☐ **Gün sonu çıktısı:** HTTP istek/yanıt akışını kendi cümlelerinizle anlatabiliyorsunuz.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Tarayıcıya bir adres yazıp Enter'a bastığınızda neler olur?
- Commit ile push arasındaki fark nedir?
- 404 ile 500 durum kodu arasındaki fark nedir? Hangisi kimin hatasıdır?

FAZ 0 · TEMELLER

Hafta 2 | Docker ile Tanışma

Haftanın hedefi: Konteyner mantığını anlamak; image, container, volume ve docker compose kavramlarını uygulamalı öğrenmek.

Çarşamba | Gün 4/48

Konteyner nedir, neden Docker?

Tarih: ____/____

ÖĞRENİN

- "Benim bilgisayarımda çalışıyordu" problemi ve Docker'ın çözümü
- Image ile container farkı; Docker Hub
- Port eşleme (port mapping) ve konteyner yaşam döngüsü

UYGULAYIN / ÖDEV

- ☐ `docker run hello-world` çalıştırınız ve çıktığı yorumlayınız
- ☐ `docker run -d -p 8080:80 nginx` ile bir web sunucusu başlatıp tarayıcıdan açınız
- ☐ `docker ps`, `docker logs`, `docker stop`, `docker rm` komutlarını deneyiniz

☐ **Gün sonu çıktısı:** Komut listesi ve ne işe yaradıkları LOG.md'de.

Perşembe | Gün 5/48

Dockerfile yazmak

Tarih: ____/____

ÖĞRENİN

- Dockerfile komutları: `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`
- Image katmanları ve build ön belleği
- Volume kavramı: konteyner silinince veri neden kaybolur?

UYGULAYIN / ÖDEV

- ☐ Kendinizi tanıtan basit bir HTML sayfası hazırlayınız
- ☐ Bu sayfayı nginx tabanlı bir Dockerfile ile paketleyip `docker build` ve `docker run` ile çalıştırınız
- ☐ Dockerfile'ı depoya commit ediniz

☐ **Gün sonu çıktısı:** Kendi image'iniz çalışıyor ve depoda.

Cuma | Gün 6/48

docker compose ve proje mimarisi

Tarih: ____/____

ÖĞRENİN

- docker compose ile birden fazla servisi birlikte yönetmek
- Servisler arası ağ (network), ortam değişkenleri (environment), volume
- StajDesk mimarisi: hangi parça neden var?

UYGULAYIN / ÖDEV

- ☐ compose dosyası ile PostgreSQL + Adminer ayağa kaldırınız; Adminer'dan veritabanına bağlanınız
- ☐ Volume tanımlayıp konteyneri silip yeniden başlatınız; verinin kaldığını gözlemleyiniz
- ☐ Bu dokümandaki mimari şemayı kâğıda kendi elinizle çizip her kutunun görevini yazınız

☐ **Gün sonu çıktısı:** Çalışan compose dosyası ve el çizimi mimari şema.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Image ile container arasındaki farkı bir benzetmeyle açıklayınız.
- Volume tanımlamasaydık veritabanı konteyneri silindiğinde ne olurdu?
- Neden her geliştiricinin bilgisayarına PostgreSQL kurmak yerine Docker kullanıyoruz?

FAZ 1 · C# VE .NET

Hafta 3 | C# Temelleri

Haftanın hedefi: C# dilinin temel yapı taşlarını öğrenmek ve basit konsol uygulamaları yazabilmek.

Çarşamba | Gün 7/48

Neden .NET? İlk C# programı

Tarih: ____/____

ÖĞRENİN

- Neden .NET: güçlü tip sistemi, yüksek performans, kurumsal ekosistem, çapraz platform
- `dotnet new console`, `dotnet run`; proje dosyası (.csproj)
- Değişkenler ve tipler: `int`, `decimal`, `string`, `bool`, `DateTime`; tip dönüşümleri

UYGULAYIN / ÖDEV

- ☐ Kullanıcıdan 3 not alıp ortalamasını ve harf notunu yazdıran konsol programı yazınız
- ☐ Hatalı giriş (harf girilmesi) durumunda programın çökmemesini sağlayınız (`int.TryParse`)

☐ **Gün sonu çıktısı:** Not hesaplayıcı depoda (/exercises/week03).

Perşembe | Gün 8/48

Kontrol akışı ve metotlar

Tarih: ____/____

ÖĞRENİN

- `if/else`, `switch`, `for`, `while`, `foreach`
- Metot tanımlama, parametreler, dönüş değeri
- Kodu okunabilir yazmak: anlamlı isimlendirme

UYGULAYIN / ÖDEV

- ☐ Sayı tahmin oyunu yazınız (1–100 arası, kaç denemede bulunduğunu gösterin)
- ☐ `AsalMi(int sayi)` metodu ve 1–100 arası asal sayıları listeleyen program
- ☐ FizzBuzz problemini çözünüz

☐ **Gün sonu çıktısı:** 3 alıştırmaya çalışır durumda ve commit'lendi.

Cuma | Gün 9/48

Koleksiyonlar ve LINQ

Tarih: ____/____

ÖĞRENİN

- Dizi, `List<T>`, `Dictionary<TKey, TValue>`
- LINQ'e giriş: `Where`, `Select`, `OrderBy`, `Count`, `FirstOrDefault`
- `null` kavramı ve null kontrolü

UYGULAYIN / ÖDEV

- ☐ 10 öğrenciden oluşan bir liste tanımlayınız (ad, sınıf, not)
- ☐ LINQ ile: notu 70 üstü olanlar, sınıfa göre ortalama, en yüksek not alan öğrenci
- ☐ Cuma değerlendirmesi: kodunuzu mentora satır satır anlatınız

☐ **Gün sonu çıktısı:** LINQ alıştırmaları tamam; her sorgunun ne yaptığını açıklayabiliyorsunuz.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- `decimal` ile `double` arasındaki fark nedir? Para için hangisi kullanılır, neden?
- `List<T>` ile dizi arasındaki fark nedir?
- LINQ olmasaydı "notu 70 üstü öğrenciler" sorgusunu nasıl yazardınız?

FAZ 1 - C# VE .NET

Hafta 4 | Nesne Yönelimli Programlama (OOP)

Haftanın hedefi: Sınıf, nesne, arayüz (interface) kavramlarını öğrenmek ve StajDesk'in ilk sürümünü konsolda yazmak.

Çarşamba | Gün 10/48**Sınıflar ve nesneler**

Tarih: ____/____

ÖĞRENİN

- Sınıf ve nesne, özellik (property), constructor
- Erişim belirleyiciler (**public**, **private**) ve kapsülleme (encapsulation)
- Değer tipi ve referans tipi farkı

UYGULAYIN / ÖDEV

- ☐ **Ticket** sınıfı oluşturunuz: **Id**, **Title**, **Description**, **Status**, **CreatedAt**
- ☐ Başlığın boş olamayacağını constructor içinde kontrol ediniz

☐ **Gün sonu çıktısı:** Ticket sınıfı ve basit testleri çalışıyor.**Perşembe** | Gün 11/48**Kalıtım, interface, enum, hata yönetimi**

Tarih: ____/____

ÖĞRENİN

- Kalıtım (inheritance) ve polimorfizm
- Interface nedir, neden kullanılır ("sözleşme" mantığı)
- enum** ve **try/catch/finally** ile hata yönetimi

UYGULAYIN / ÖDEV

- ☐ **TicketStatus** enum'u tanımlayınız: **Open**, **InProgress**, **Resolved**, **Closed**
- ☐ **ITicketRepository** interface'i ve bunu uygulayan **InMemoryTicketRepository** sınıfını yazınız
- ☐ Olmayan bir Id istendiğinde anlamlı bir hata fırlatınız

☐ **Gün sonu çıktısı:** Repository katmanı konsolda çalışıyor.**Cuma** | Gün 12/48**async/await ve Konsol Mini Talep Yöneticisi**

Tarih: ____/____

ÖĞRENİN

- Senkron ve asenkron çalışma; **Task**, **async**, **await** (neden beklerken bloklamayız?)
- Konsol menüsü tasarımı

UYGULAYIN / ÖDEV

- ☐ Menüli konsol uygulaması: talep ekle, listele, durum güncelle, sil
- ☐ Repository metotlarını **async** hâle getiriniz
- ☐ Cuma: mentor kod incelemesi; aldığınız geri bildirimleri uygulayınız

☐ **Gün sonu çıktısı:** StajDesk v0 (konsol) depoda.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Interface kullanmanın bize ne faydası var? **ITicketRepository** örneği üzerinden anlatınız.
- Kapsülleme neden önemlidir?
- await** kullanmasaydık web sunucusunda ne gibi bir sorun olurdu?

FAZ 1 · C# VE .NET

Hafta 5 | ASP.NET Core Web API**Haftanın hedefi:** Konsol uygulamasını, HTTP üzerinden erişilebilen bir REST API'ye dönüştürmek.**Çarşamba** | Gün 13/48**Web API'ye giriş**

Tarih: ____/____

ÖĞRENİN

- API nedir; REST prensipleri (kaynak, URL, metod)
- `dotnet new webapi`; `Program.cs`, middleware pipeline
- Swagger/OpenAPI ile API'yi keşfetmek

UYGULAYIN / ÖDEV

- ☐ `StajDesk.Api` projesini oluşturunuz
- ☐ `GET /api/health` endpoint'i yazınız (sunucu saati ve "OK" döner)
- ☐ Swagger arayüzünden deneyiniz

☐ **Gün sonu çıktısı:** API ayağa kalkıyor, Swagger açılıyor.**Perşembe** | Gün 14/48**Controller, routing ve Dependency Injection**

Tarih: ____/____

ÖĞRENİN

- Controller, route, model binding, `ActionResult`
- Dependency Injection nedir; `Singleton`, `Scoped`, `Transient` farkı
- Doğru durum kodlarını dönmek (200, 201, 204, 404)

UYGULAYIN / ÖDEV

- ☐ `TicketsController` yazınız: GET (liste), GET /{id}, POST, PUT, DELETE
- ☐ Geçen haftaki `InMemoryTicketRepository`'yi DI ile kaydedip kullanınız

☐ **Gün sonu çıktısı:** CRUD uçları Swagger'dan çalışıyor.**Cuma** | Gün 15/48**Katmanlı mimari ve DTO**

Tarih: ____/____

ÖĞRENİN

- Controller → Service → Repository katmanları; controller'da iş mantığı olmamalı
- DTO (Data Transfer Object) nedir; neden entity'yi doğrudan dışarı açmayız?

UYGULAYIN / ÖDEV

- ☐ `TicketService` oluşturup iş kurallarını buraya taşıyınız
- ☐ `CreateTicketRequest` ve `TicketResponse` DTO'larını yazınız
- ☐ Postman koleksiyonu hazırlayıp depoya ekleyiniz

☐ **Gün sonu çıktısı:** Katmanlı API + Postman koleksiyonu depoda.**Kavrama soruları** (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Uygulamayı yeniden başlattığınızda talepler neden kayboluyor?
- Dependency Injection olmasaydı `TicketsController` nasıl görünürdü?
- POST başarılı olduğunda neden 200 yerine 201 döneriz?

FAZ 2 · POSTGRESQL

Hafta 6 | İlişkisel Veritabanı ve SQL

Haftanın hedefi: Verinin kalıcı olarak nasıl saklandığını öğrenmek, SQL yazabilmek ve StajDesk veri modelini tasarlamak.

Çarşamba | Gün 16/48

Veritabanı nedir, neden PostgreSQL?

Tarih: ____/____

ÖĞRENİN

- İlişkisel model: tablo, satır, sütun, birincil anahtar (primary key)
- Neden PostgreSQL: açık kaynak, güvenilir (ACID), zengin veri tipleri, yaygın kullanım
- Temel komutlar: **CREATE TABLE**, **INSERT**, **SELECT**

UYGULAYIN / ÖDEV

- ☐ Docker'daki PostgreSQL'e **psql** ile bağlanınız
- ☐ **categories** tablosunu elle oluşturup 5 kayıt ekleyiniz

☐ **Gün sonu çıktısı:** İlk tablo ve kayıtlar hazır.

Perşembe | Gün 17/48

SQL sorguları ve ilişkiler

Tarih: ____/____

ÖĞRENİN

- WHERE**, **ORDER BY**, **UPDATE**, **DELETE**
- Yabancı anahtar (foreign key), **JOIN**, **GROUP BY**, **COUNT**

UYGULAYIN / ÖDEV

- ☐ **users**, **tickets**, **comments** tablolarını ilişkileriyle oluşturunuz; 20 örnek kayıt giriniz
- ☐ Mentorun vereceği 10 soruya SQL ile cevap yazınız (ör. kategoriye göre açık talep sayısı)
- ☐ Sorguları **/sql/week06.sql** dosyasına kaydediniz

☐ **Gün sonu çıktısı:** 10 SQL sorgusu depoda.

Cuma | Gün 18/48

Veri modelleme

Tarih: ____/____

ÖĞRENİN

- ER (varlık-ilişki) diyagramı, 1-N ilişkiler
- Normalizasyona giriş; index nedir, neyi hızlandırır, bedeli nedir?

UYGULAYIN / ÖDEV

- ☐ StajDesk ER diyagramını çiziniz (draw.io veya kâğıt): User, Category, Ticket, Comment, Notification
- ☐ Hangi sütunlara index koyacağınızı gerekçesiyle yazınız
- ☐ Cuma: diyagramı mentorla gözden geçiriniz

☐ **Gün sonu çıktısı:** Onaylanmış ER diyagramı depoda (**/docs**).**Kavrama soruları** (cuma değerlendirmesinden önce **LOG.md**'ye kendi cümlelerinizle cevaplayınız)

- Talepleri bir metin dosyasında değil de veritabanında tutmamızın 3 nedeni nedir?
- JOIN** ne işe yarar? Kendi tablolarınızdan bir örnek veriniz.
- Her sütuna index koymak neden iyi bir fikir değildir?

FAZ 2 · POSTGRESQL

Hafta 7 | Entity Framework Core ile API'yi Veritabanına Bağlamak**Haftanın hedefi:** ORM kavramını öğrenmek ve API'nin verisini PostgreSQL'de kalıcı hâle getirmek.**Çarşamba** | Gün 19/48**ORM ve EF Core**

Tarih: ____/____

ÖĞRENİN

- ORM nedir; SQL yazmak ile ORM kullanmak arasındaki fark
- EF Core, Npgsql sağlayıcısı, **DbContext**, entity sınıfları
- Bağlantı cümlesi (connection string) ve **appsettings.json**

UYGULAYIN / ÖDEV

- ☐ Gerekli NuGet paketlerini ekleyiniz
- ☐ **AppDbContext** ve entity'leri (User, Category, Ticket, Comment) yazınız

☐ **Gün sonu çıktısı:** Proje derleniyor, DbContext hazır.**Perşembe** | Gün 20/48**Migration ve ilişkiler**

Tarih: ____/____

ÖĞRENİN

- Migration nedir, neden elle tablo oluşturmayız?
- Navigation property, Fluent API ile yapılandırma

UYGULAYIN / ÖDEV

- ☐ İlk migration'ı oluşturup veritabanına uygulayınız
- ☐ Tabloların **psql** ile oluşturulduğunu doğrulayınız
- ☐ Varsayılan kategorileri seed ediniz

☐ **Gün sonu çıktısı:** Migration'lar depoda, tablolar oluştu.**Cuma** | Gün 21/48**Repository'yi EF Core'a taşımak**

Tarih: ____/____

ÖĞRENİN

- Async sorgular, **Include**, **AsNoTracking**
- Sayfalama (**Skip/Take**) ve filtreleme

UYGULAYIN / ÖDEV

- ☐ **EfTicketRepository** yazıp DI kaydını değiştiriniz (interface sayesinde controller değişmez!)
- ☐ **GET /api/tickets?status=open&categoryId=2&page=1&pageSize=10** desteği ekleyiniz
- ☐ Cuma: API'yi yeniden başlatıp verinin kaldığını mentora gösteriniz

☐ **Gün sonu çıktısı:** API artık kalıcı veriyle çalışıyor.**Kavrama soruları** (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Repository'yi değiştirdiğimizde controller'ın değişmemesi neyi kanıtlıyor?
- Migration olmadan ekte 5 kişi aynı veritabanı şemasını nasıl senkron tutardı?
- Sayfalama yapmasaydık 1 milyon talep olduğunda ne olurdu?

FAZ 3 · GÜVENLİK

Hafta 8 | Doğrulama, Hata Yönetimi ve Kimlik Doğrulama

Haftanın hedefi: API'yi hatalı girdiye ve yetkisiz erişime karşı korumak; kayıt, giriş ve rol yönetimini eklemek.

Çarşamba | Gün 22/48

Doğrulama ve hata yönetimi

Tarih: ____/____/____

ÖĞRENİN

- Neden doğrulama her zaman sunucu tarafında yapılır?
- DataAnnotations veya FluentValidation ile doğrulama
- Global hata yakalama middleware'i ve standart yanıt zarfı (**success**, **data**, **message**)

UYGULAYIN / ÖDEV

- ☐ Tüm request DTO'larına doğrulama kuralları ekleyiniz
- ☐ Tüm uçların aynı yanıt formatını dönmesini sağlayınız

☐ **Gün sonu çıktısı:** Hatalı istekler anlaşılır 400 yanıtı alıyor.

Perşembe | Gün 23/48

Kayıt, giriş ve JWT

Tarih: ____/____/____

ÖĞRENİN

- Kimlik doğrulama (authentication) ile yetkilendirme (authorization) farkı
- Şifreler neden düz metin saklanmaz; hash ve salt kavramı
- JWT'nin yapısı: header, payload, signature

UYGULAYIN / ÖDEV

- ☐ **POST /api/auth/register** ve **POST /api/auth/login** uçlarını yazınız
- ☐ Şifreleri hash'leyerek saklayınız; login başarılıysa JWT döndürünüz
- ☐ Gizli anahtarları koda değil **user-secrets** / **.env** içine koyunuz

☐ **Gün sonu çıktısı:** Kayıt ve giriş çalışıyor; depoda hiçbir gizli anahtar yok.

Cuma | Gün 24/48

Roller ve yetkilendirme

Tarih: ____/____/____

ÖĞRENİN

- [**Authorize**] ve rol bazlı yetki: **User**, **Agent** (destek personeli), **Admin**
- Kullanıcı yalnızca kendi taleplerini görmeli; Agent tüm talepleri görebilmeli

UYGULAYIN / ÖDEV

- ☐ Yetki kurallarını uygulayınız ve yorum ekleme ucunu (**POST /api/tickets/{id}/comments**) yazınız
- ☐ Postman'da token ile istek atmayı koleksiyona ekleyiniz
- ☐ Cuma: başka bir kullanıcının talebine erişmeyi deneyip engellendiğini gösteriniz

☐ **Gün sonu çıktısı:** Güvenli ve rol bazlı çalışan API.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- JWT'nin içeriği okunabiliyorsa neden içine şifre veya gizli bilgi koymayız?
- Frontend'de doğrulama yapıyorsak backend'de neden tekrar yapıyoruz?
- 401 ile 403 arasındaki fark nedir?

FAZ 4 · VUE.JS

Hafta 9 | Frontend Temelleri ve Vue.js'e Giriş

Haftanın hedefi: Tarayıcı tarafında neler olduğunu anlamak ve Vue'nun neden kullanıldığını deneyimleyerek görmek.

Çarşamba | Gün 25/48**HTML/CSS/JavaScript tekrarı**

Tarih: ____/____

ÖĞRENİN

- DOM, olaylar (event), **fetch**, Promise
- Neden Vue? Bileşen yapısı, reaktivite, öğrenme kolaylığı, tek sayfa uygulama (SPA)

UYGULAYIN / ÖDEV

- ☐ Saf JavaScript ile API'den talepleri çekip listeleyen tek bir HTML sayfası yazınız
- ☐ Aynı sayfaya filtre eklemeye çalışınız ve ne kadar zorlaştığını LOG.md'ye not ediniz

☐ **Gün sonu çıktısı:** Vanilla JS sayfası çalışıyor; zorluklar not edildi.**Perşembe** | Gün 26/48**Vue 3 temelleri**

Tarih: ____/____

ÖĞRENİN

- Vite ile proje oluşturma; **.vue** dosya yapısı
- Composition API: **ref**, **reactive**, **computed**
- Direktifler: **v-if**, **v-for**, **v-model**, **@click**

UYGULAYIN / ÖDEV

- ☐ **stajdesk-web** projesini oluşturunuz
- ☐ Sayaç ve yapılacaklar listesi (to-do) bileşenleri yazınız

☐ **Gün sonu çıktısı:** Vue projesi çalışıyor, iki alıştırma tamam.**Cuma** | Gün 27/48**Bileşenler**

Tarih: ____/____

ÖĞRENİN

- Bileşen, **props**, **emit**, **slot**
- Klasör yapısı: **views**, **components**, **services**, **stores**

UYGULAYIN / ÖDEV

- ☐ **TicketCard** ve **StatusBadge** bileşenlerini yazınız
- ☐ Sahte (mock) veriyle talep listesi sayfası hazırlayınız
- ☐ Cuma: pazartesi yazdığınız vanilla JS ile Vue sürümünü karşılaştırıp anlatınız

☐ **Gün sonu çıktısı:** Bileşen tabanlı liste sayfası.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

1. Reaktivite nedir? Vue olmadan aynı etkiyi nasıl elde ederdiniz?
2. **props** ile **emit** arasındaki veri akışını çizerek anlatınız.
3. Neden sayfayı tek bir büyük dosya yerine bileşenlere böleriz?

FAZ 4 · VUE.JS

Hafta 10 | Vue Router, Pinia ve API Bağlantısı**Haftanın hedefi:** Frontend'i gerçek API'ye bağlamak; sayfa yönlendirme ve oturum yönetimini kurmak.

Çarşamba Gün 28/48	Vue Router	Tarih: ____/____
ÖĞRENİN - Sayfa yönlendirme, dinamik route (/tickets/:id) - Navigation guard ile sayfa koruma	UYGULAYIN / ÖDEV ☐ Sayfa iskeletlerini oluşturunuz: Giriş, Kayıt, Taleplerim, Talep Detayı, Yeni Talep	
☐ Gün sonu çıktısı: Sayfalar arası gezinme çalışıyor.		

Perşembe Gün 29/48	API bağlantısı ve CORS	Tarih: ____/____
ÖĞRENİN - Axios ile istek atma; servis katmanı (services/api.js) - CORS nedir, neden tarayıcı engeller; API'de nasıl ayarlanır - Ortam değişkenleri (.env, VITE_API_URL)	UYGULAYIN / ÖDEV ☐ Kayıt ve giriş formlarını API'ye bağlayınız ☐ Hata mesajlarını kullanıcıya resmî dille gösteriniz (ör. "E-posta adresiniz veya şifreniz hatalıdır.")	
☐ Gün sonu çıktısı: Frontend üzerinden kayıt ve giriş yapılabilir.		

Cuma Gün 30/48	Pinia ile oturum yönetimi	Tarih: ____/____
ÖĞRENİN - State management nedir; Pinia store - Axios interceptor ile her isteğe token eklemek; 401'de oturumu kapatmak	UYGULAYIN / ÖDEV ☐ auth store'u yazınız (token, kullanıcı, rol) ☐ Giriş yapmamış kullanıcıyı korumalı sayfalardan Giriş sayfasına yönlendiriniz ☐ Cuma: uçtan uca giriş akışını gösteriniz	
☐ Gün sonu çıktısı: Oturum yönetimi tamam.		

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- CORS hatası neden Postman'da değil de tarayıcıda çıkar?
- Token'ı her bileşende ayrı ayrı tutmak yerine neden store kullanıyoruz?
- Frontend'deki route guard güvenlik için yeterli midir? Neden?

FAZ 4 · VUE.JS

Hafta 11 | Uygulama Ekranları

Haftanın hedefi: StajDesk'in tüm kullanıcı ekranlarını tamamlamak; uygulamayı uçtan uca kullanılabilir hâle getirmek.

Çarşamba | Gün 31/48**Talep listesi**

Tarih: ____/____

ÖĞRENİN

- Liste ekranı tasarımı: filtre, sayfalama
- Yükleniyor, boş liste ve hata durumlarının kullanıcıya gösterimi

UYGULAYIN / ÖDEV

- ☐ Taleplerim sayfasını API'ye bağlayınız; durum ve kategori filtresi, sayfalama ekleyiniz
- ☐ Üç durumun (yükleniyor/boş/hata) her birini ekranda gösteriniz

☐ **Gün sonu çıktısı:** Talep listesi tamam.**Perşembe** | Gün 32/48**Talep detayı ve yeni talep**

Tarih: ____/____

ÖĞRENİN

- Form doğrulama ve kullanıcı dostu hata mesajları
- Rol bazlı arayüz: Agent'ın görebildiği ek butonlar

UYGULAYIN / ÖDEV

- ☐ Talep detayı: açıklama, yorumlar, yorum ekleme
- ☐ Agent rolü için durum değiştirme
- ☐ Yeni talep formu (başlık, açıklama, kategori) ve doğrulama

☐ **Gün sonu çıktısı:** Talep oluşturma ve takip akışı tamam.**Cuma** | Gün 33/48**Yönetim ekranı ve özet paneli**

Tarih: ____/____

ÖĞRENİN

- Basit bir özet paneli (dashboard) için API tasarımı
- Responsive (mobil uyumlu) düzen

UYGULAYIN / ÖDEV

- ☐ `GET /api/stats` ucu: duruma göre talep sayıları
- ☐ Admin için kategori yönetimi ve özet paneli ekranı
- ☐ Cuma DEMO: kayıt → talep aç → Agent yanıtlasın → talep kapansın akışını gösteriniz

☐ **Gün sonu çıktısı:** StajDesk uçtan uca çalışıyor (MVP).

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Uygulamanın şu anki hâlinde performans sorunu çıkabilecek yerler nereler?
- Özet paneli her açıldığında veritabanında hangi sorgular çalışıyor?
- Kullanıcı deneyimi açısından en çok neyi geliştirmek isterdiniz?

FAZ 5 · REDİS

Hafta 12 | Redis ile Önbellekleme ve Hız Sınırlama

Haftanın hedefi: Redis'in ne olduğunu, veritabanından farkını ve neden kullanıldığını ölçerek öğrenmek.

Çarşamba | Gün 34/48**Redis nedir?**

Tarih: ____/____

ÖĞRENİN

- Bellek içi (in-memory) anahtar-değer deposu; neden bu kadar hızlı?
- Redis neden PostgreSQL'in yerini tutmaz, yanında kullanılır?
- Veri tipleri: string, hash, list, set, sorted set; TTL (yaşam süresi)

UYGULAYIN / ÖDEV

- ☐ compose dosyasına Redis ekleyiniz
- ☐ `redis-cli` ile **SET**, **GET**, **EXPIRE**, **TTL**, **INCR**, **DEL** komutlarını deneyiniz
- ☐ Her komutun ne yaptığını `LOG.md`'ye örneğiyle yazınız

☐ **Gün sonu çıktısı:** Redis çalışıyor, komut notları hazır.

Perşembe | Gün 35/48**Önbellekleme (cache-aside)**

Tarih: ____/____

ÖĞRENİN

- Cache-aside deseni: önce önbelleğe bak, yoksa veritabanından al ve önbelleğe yaz
- Önbellek geçersiz kılma (cache invalidation): veri değişince ne olmalı?
- StackExchange.Redis** veya **IDistributedCache** kullanımı

UYGULAYIN / ÖDEV

- ☐ Kategori listesi ve `/api/stats` sonuçlarını 60 saniyeliğine önbelleğe alınız
- ☐ Kategori eklendiğinde/değiştirildiğinde ilgili önbelleği siliniz
- ☐ **Stopwatch** ile önbellekli ve önbelleksiz süreleri ölçüp `LOG.md`'ye yazınız

☐ **Gün sonu çıktısı:** Ölçümlerle kanıtlanmış önbellekleme.

Cuma | Gün 36/48**Hız sınırlama (rate limiting)**

Tarih: ____/____

ÖĞRENİN

- Brute-force saldırısı nedir?
- Redis **INCR** + **EXPIRE** ile sayaç tabanlı hız sınırlama

UYGULAYIN / ÖDEV

- ☐ Aynı e-posta ile 5 dakikada en fazla 5 hatalı giriş denemesine izin veriniz
- ☐ Limit aşıldı 429 dönünüz; frontend'de "Çok fazla deneme yaptınız, lütfen birkaç dakika sonra tekrar deneyiniz." mesajı gösteriniz
- ☐ Cuma: Redis konteynerini durdurup uygulamanın davranışını gözlemleyiniz ve mentorla tartışınız

☐ **Gün sonu çıktısı:** Giriş ucu brute-force'a karşı korumalı.

Kavrama soruları (cuma değerlendirmesinden önce `LOG.md`'ye kendi cümlelerinizle cevaplayınız)

- Redis'i ana veritabanı olarak kullansaydık ne gibi riskler olurdu?
- Önbellekteki veri eskidiğinde (stale) kullanıcı ne görür? Bu kabul edilebilir mi?
- Redis kapanırsa uygulama tamamen durmalı mı, yoksa veritabanıyla devam mı etmeli?

FAZ 6 · RABBITMQ

Hafta 13 | RabbitMQ ile Asenkron Mesajlaşma

Haftanın hedefi: Mesaj kuyruğu mantığını öğrenmek; bildirimleri API'den bağımsız çalışan bir arka plan servisine taşımak.

Çarşamba | Gün 37/48**Mesaj kuyruğu nedir?**

Tarih: ____/____

ÖĞRENİN

- Senkron ve asenkron iletişim; işleri birbirinden ayırmak (decoupling)
- Producer, consumer, queue, exchange, routing key
- Neden RabbitMQ: yük altında dayanıklılık, mesaj kaybetmeme, ölçeklenebilirlik

UYGULAYIN / ÖDEV

- ☐ compose dosyasına `rabbitmq:management` ekleyiniz; yönetim arayüzünü (15672) açınız
- ☐ Arayüzden elle kuyruk oluşturup mesaj yayınlayınız ve okuyunuz
- ☐ RabbitMQ resmi "Hello World" öğreticisini C# ile tamamlayınız

☐ **Gün sonu çıktısı:** İlk producer/consumer çalışıyor.**Perşembe** | Gün 38/48**Olay yayınlama ve Worker servisi**

Tarih: ____/____

ÖĞRENİN

- Olay (event) tasarımı: `TicketCreated`, `TicketStatusChanged`
- Worker Service (`dotnet new worker`), `BackgroundService`
- Kalıcı kuyruk (durable), mesaj onayı (ack/nack)

UYGULAYIN / ÖDEV

- ☐ API'de talep oluşturulduğunda ve durum değiştiğinde olayı JSON olarak yayınlayınız
- ☐ `StajDesk.Worker` projesini oluşturup bu olayları dinleyiniz

☐ **Gün sonu çıktısı:** API olay yayınlıyor, Worker dinliyor.**Cuma** | Gün 39/48**Bildirimler**

Tarih: ____/____

ÖĞRENİN

- Worker'ın veritabanına yazması; bildirim modeli
- Frontend'de periyodik yenileme (polling)

UYGULAYIN / ÖDEV

- ☐ Worker gelen olaylardan `notifications` tablosuna kayıt oluştursun ve "e-posta gönderildi" logu yazsın
- ☐ `GET /api/notifications` ucu ve Vue'da bildirim zili
- ☐ Deney: Worker'ı durdurun, 3 talep açın, Worker'ı başlatın; mesajların kaybolmadığını gösteriniz

☐ **Gün sonu çıktısı:** Asenkron bildirim sistemi çalışıyor.**Kavrama soruları** (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Bildirimi doğrudan API içinde göndermek yerine neden kuyruğa atıyoruz?
- Worker kapalıyken gelen mesajlara ne oldu? Bu neden önemli?
- Mesaj işlenirken hata olursa ack göndermezsek ne olur?

FAZ 7 · BÜTÜNLEŞTİRME

Hafta 14 | Tüm Sistemin Docker ile Çalıştırılması

Haftanın hedefi: Tek bir komutla (`docker compose up`) tüm sistemin herhangi bir bilgisayarda ayağa kalkmasını sağlamak.

Çarşamba | Gün 40/48

Backend Dockerfile'ları

Tarih: ____/____

ÖĞRENİN

- Çok aşamalı (multi-stage) build: SDK ile derle, runtime ile çalıştır
- `.dockerignore`; image boyutunu küçültmek

UYGULAYIN / ÖDEV

- ☐ API ve Worker için Dockerfile yazınız
- ☐ Image boyutlarını karşılaştırıp not ediniz (SDK vs runtime)

☐ **Gün sonu çıktısı:** API ve Worker konteynerde çalışıyor.

Perşembe | Gün 41/48

Frontend Dockerfile'ı

Tarih: ____/____

ÖĞRENİN

- Vue projesini build etmek; statik dosyaları nginx ile sunmak
- nginx ile `/api` isteklerini backend'e yönlendirmek (reverse proxy)

UYGULAYIN / ÖDEV

- ☐ Frontend Dockerfile'ı ve `nginx.conf` yazınız

☐ **Gün sonu çıktısı:** Frontend konteynerde çalışıyor.

Cuma | Gün 42/48

Tek compose dosyası

Tarih: ____/____

ÖĞRENİN

- `depends_on` ve `healthcheck`; servislerin başlama sırası
- Ortam değişkenleri ve `.env` dosyası (`.env` depoya girmez, `.env.example` girer)

UYGULAYIN / ÖDEV

- ☐ postgres, redis, rabbitmq, api, worker, web servislerini tek compose dosyasında birleştiriniz
- ☐ API açılışında migration ve seed otomatik çalışsın
- ☐ Cuma: depoyu temiz bir bilgisayara klonlayıp `docker compose up` ile çalıştırınız

☐ **Gün sonu çıktısı:** Tek komutla ayağa kalkan sistem.

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Multi-stage build neden image boyutunu küçültür?
- `depends_on` tek başına servisin hazır olduğunu garanti eder mi?
- `.env` dosyasını neden depoya koymuyoruz?

FAZ 7 · BÜTÜNLEŞTİRME

Hafta 15 | Kalite: Test, Loglama ve Dokümantasyon

Haftanın hedefi: Yazılan kodun doğruluğunu test etmek, sorunları izlenebilir kılmak ve projeyi belgelemek.

Çarşamba Gün 43/48	Birim testler	Tarih: ____/____
ÖĞRENİN - Birim test nedir, neden yazılır; Arrange-Act-Assert - xUnit ile test projesi	UYGULAYIN / ÖDEV ☐ TicketService için en az 8 birim test yazınız (geçerli/geçersiz talep, yetki, durum geçişleri)	
☐ Gün sonu çıktısı: Testler yeşil (dotnet test).		

Perşembe Gün 44/48	Loglama ve güvenlik kontrolü	Tarih: ____/____
ÖĞRENİN - ILogger, log seviyeleri (Information, Warning, Error) - Loga ne yazılır, ne yazılmaz (şifre, token asla)	UYGULAYIN / ÖDEV ☐ Önemli noktalara anlamlı loglar ekleyiniz ☐ Güvenlik kontrol listesini tamamlayınız: depoda gizli anahtar yok, şifreler hash'li, tüm uçlarda yetki ve doğrulama var	
☐ Gün sonu çıktısı: Kontrol listesi tamamlandı.		

Cuma Gün 45/48	Dokümantasyon ve kod incelemesi	Tarih: ____/____
ÖĞRENİN - İyi bir README nasıl olur - Kod incelemesinden (code review) nasıl faydalanılır	UYGULAYIN / ÖDEV ☐ README: kurulum, mimari şema, teknolojilerin görevleri, ekran görüntüleri ☐ Cuma: mentorla kapsamlı kod incelemesi; notları bir sonraki haftaya iş olarak yazınız	
☐ Gün sonu çıktısı: Belgelenmiş, test edilmiş proje.		

Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)

- Test yazmak zaman kaybı mıdır? Kendi projenizden bir örnek ile cevaplayınız.
- Bir hata canlı ortamda çıksaydı loglarınız sebebini bulmaya yeter miydi?
- Projeyi ilk kez gören biri README ile sistemi çalıştırabilir mi?

FAZ 8 · KAPANIŞ

Hafta 16 | Son Dokunuşlar ve Final Sunumu**Haftanın hedefi:** Projeyi tamamlamak, öğrendiklerinizi ekibe anlatmak ve staj sürecini değerlendirmek.**Çarşamba | Gün 46/48** **Eksikleri tamamlama**

Tarih: ____/____/____

ÖĞRENİN

- Önceliklendirme: hangi hata önce düzeltilmeli?

UYGULAYIN / ÖDEV

- ☐ Kod incelemesinden gelen notları kapatınız
- ☐ Zaman kalırsa "Bonus Görevler" listesinden birini seçiniz

☐ **Gün sonu çıktısı:** Açık hata listesi boş.**Perşembe | Gün 47/48** **Sunum hazırlığı**

Tarih: ____/____/____

ÖĞRENİN

- Teknik bir konuyu teknik olmayan birine anlatmak

UYGULAYIN / ÖDEV

- ☐ 10–15 slaytlık sunum: problem, mimari, her teknolojinin görevi ve "neden"i, demo senaryosu, öğrendikleriniz
- ☐ Mentorla prova yapınız

☐ **Gün sonu çıktısı:** Sunum ve demo senaryosu hazır.**Cuma | Gün 48/48** **Final sunumu ve değerlendirme**

Tarih: ____/____/____

ÖĞRENİN

- Geri bildirim almak ve vermek

UYGULAYIN / ÖDEV

- ☐ Ekibe 20 dakikalık sunum + soru-cevap
- ☐ Retrospektif: neler iyi gitti, neler zorladı, bundan sonra ne öğrenmek istiyorsunuz?

☐ **Gün sonu çıktısı:** Staj tamamlandı.**Kavrama soruları (cuma değerlendirmesinden önce LOG.md'ye kendi cümlelerinizle cevaplayınız)**

- Bu projeyi baştan yapsaydınız neyi farklı yapardınız?
- 5 teknolojinin her birini bir cümleyle, teknik olmayan birine anlatınız.
- Bundan sonraki öğrenme hedefiniz nedir?

Bonus Görevler

Planın önünde ilerlerseniz veya son haftada zamanınız kalırsa aşağıdakilerden birini mentorunuzla birlikte seçebilirsiniz:

- Talebe dosya eki yükleme (boyut ve tür sınırlamasıyla)
- SignalR ile gerçek zamanlı bildirim (sayfayı yenilemeden)
- MailHog konteyneri ile Worker'dan gerçek test e-postası gönderme
- Talep başlığı ve açıklamasında arama (PostgreSQL tam metin arama)
- Talebe öncelik alanı ve öncelik bazlı sıralama
- Açık/koyu tema desteği

Kaynaklar

Tüm kaynaklar ücretsizdir. Öncelik her zaman resmî dokümantasyondur.

Konu	Kaynak
Git	git-scm.com/book (Pro Git kitabı, Türkçe çevirisi mevcuttur)
Web / HTTP / JS	developer.mozilla.org (MDN Web Docs)
Docker	docs.docker.com/get-started
C# ve .NET	learn.microsoft.com/dotnet/csharp (C# eğitimleri)
ASP.NET Core	learn.microsoft.com/aspnet/core (Web API öğreticileri)
EF Core	learn.microsoft.com/ef/core
PostgreSQL	postgresql.org/docs (Tutorial bölümü)
Vue.js	vuejs.org (Guide ve Tutorial) · router.vuejs.org · pinia.vuejs.org
Redis	redis.io/docs
RabbitMQ	rabbitmq.com/tutorials (C# öğreticileri 1–5)

Değerlendirme

Staj sonunda değerlendirme yalnızca bitmiş uygulamaya değil, süreç boyunca gösterdiğiniz gelişime göre yapılır.

Kriter	Ağırlık	Neye bakılır?
Kavrayış	%30	Her teknolojinin ne olduğunu ve neden kullanıldığını açıklayabilme; kavrama sorularına verilen cevaplar
Çalışan uygulama	%25	Planlanan özelliklerin çalışması; <code>docker compose up</code> ile sistemin ayağa kalkması
Kod kalitesi	%15	Okunabilirlik, katmanlı yapı, isimlendirme, testler
Düzen ve disiplin	%15	Düzenli commit'ler, eksiksiz LOG.md, zamanında teslim
İletişim	%15	Soru sorma, geri bildirimi uygulama, cuma değerlendirmeleri ve final sunumu

Son söz

Takılmak, hata yapmak ve aynı şeyi birkaç kez denemek öğrenmenin doğal parçasıdır. Önemli olan, her takıldığınızda nedenini anlamaya çalışmanız ve bunu günlüğünüze yazmanızdır. Başarılar dileriz.